

Fase 0 — Planeación y Diseño del Laboratorio de Bases de Datos Distribuidas

Laboratorio académico construido sobre **una sola computadora física**, con **Oracle VirtualBox** y **Ubuntu Server LTS + MariaDB**. Sin Docker, sin Kubernetes.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar qué es un sistema de bases de datos distribuidas y diferenciarlo de un sistema centralizado.
 2. Identificar los temas que se cubrirán a lo largo del laboratorio (replicación física, lógica, fragmentación horizontal/vertical/híbrida, consultas distribuidas, transparencia, tolerancia a fallos).
 3. Diseñar la arquitectura objetivo final del laboratorio antes de tocar una sola máquina virtual.
 4. Calcular los requisitos de hardware del equipo anfitrión (host) necesarios para soportar el crecimiento gradual del laboratorio.
 5. Definir una convención de nombres, un esquema de direccionamiento IP y un plan de crecimiento por fases.
 6. Justificar por qué la planeación previa reduce el retrabajo en fases posteriores.
-

B. Conceptos teóricos necesarios

1. Sistema de bases de datos distribuidas (SBDD). Conjunto de nodos interconectados, cada uno con su propio motor de base de datos, que se presentan ante el usuario como si fueran un único sistema lógico.

2. Replicación física vs. lógica.

- *Física:* se replican los cambios a nivel de bloques/eventos binarios del motor de almacenamiento (en MariaDB, vía binlog row-based o las posiciones GTID). Es la base de la alta disponibilidad clásica maestro-esclavo.
- *Lógica:* se replican declaraciones SQL o eventos lógicos interpretables (statement-based replication, o herramientas de replicación multi-maestro como Galera). Permite topologías más flexibles (multi-maestro, bidireccional).

3. Fragmentación (particionamiento de datos).

- *Horizontal*: se dividen las filas de una tabla entre varios nodos (ej. clientes del norte en un nodo, del sur en otro).
- *Vertical*: se dividen las columnas de una tabla entre varios nodos (ej. datos personales en un nodo, datos financieros en otro).
- *Híbrida*: combinación de ambas estrategias.

4. Transparencia de distribución. Capacidad del sistema de ocultar al usuario/aplicación el hecho de que los datos están distribuidos: transparencia de fragmentación, de ubicación, de replicación y de concurrencia.

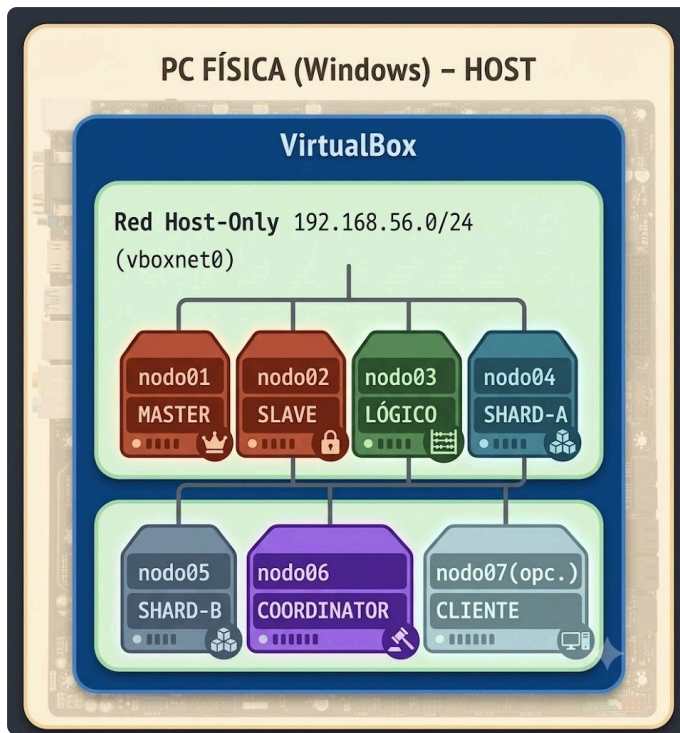
5. Teorema CAP. En presencia de una partición de red (P), un sistema distribuido debe elegir entre Consistencia © y Disponibilidad (A). Este laboratorio nos permitirá *observar* este compromiso de forma práctica, no solo teórica.

6. Tolerancia a fallos y recuperación. Capacidad del sistema de seguir operando (o de recuperarse limpiamente) ante la caída de uno o varios nodos.

C. Arquitectura utilizada

C.1 Arquitectura física (la que existe hoy)

Una sola laptop/PC con Windows ejecuta VirtualBox. Cada “nodo” del sistema distribuido es, en realidad, una máquina virtual Ubuntu Server dentro de ese mismo equipo. Una red virtual interna (Host-Only) interconecta las VMs entre sí y con el host, simulando una red de centro de datos.



C.2 Arquitectura lógica objetivo (a dónde llegaremos, gradualmente)

Nodo	Hostname propuesto	Rol funcional (fase en que se activa)
1	<code>bdd-nodo01</code>	Maestro de replicación física (Fase 10)
2	<code>bdd-nodo02</code>	Esclavo de replicación física (Fase 10)
3	<code>bdd-nodo03</code>	Nodo de replicación lógica / multi-maestro (Fase 11)
4	<code>bdd-nodo04</code>	Fragmento horizontal A (Fase 13)
5	<code>bdd-nodo05</code>	Fragmento horizontal B (Fase 13)
6	<code>bdd-nodo06</code>	Coordinador de consultas distribuidas / fragmentación vertical-híbrida (Fases 14-16)
7 (opcional)	<code>bdd-cliente</code>	Estación cliente externa para pruebas de transparencia (Fase 16-17)

Importante: no crearemos los 7 nodos ahora. Esta tabla es el mapa final. El laboratorio crecerá de 2 nodos (Fase 3) hasta 6-7 nodos (Fase 16), apagando con snapshots los nodos que no se usen en cada sesión para ahorrar recursos del host.

C.3 Plan de direccionamiento IP (reservado para cuando configuremos la red)

Nodo	IP planeada	Tipo de red
Host (Windows)	192.168.56.1	Host-Only Adapter (vboxnet0)
bdd-nodo01	192.168.56.101	Estática
bdd-nodo02	192.168.56.102	Estática
bdd-nodo03	192.168.56.103	Estática
bdd-nodo04	192.168.56.104	Estática
bdd-nodo05	192.168.56.105	Estática
bdd-nodo06	192.168.56.106	Estática
bdd-cliente	192.168.56.107	Estática

Se elige **Host-Only Adapter** (no NAT, no Bridged) porque: aísla el laboratorio de la red física/Wi-Fi real (seguridad y evita interferir con la red del salón), y permite que el host (Windows) y todas las VMs se vean entre sí, lo cual facilita administración con SSH, DBeaver, etc. Esta decisión se justificará a fondo en la Fase 2.

C.4 Dimensionamiento de recursos por nodo (referencia para Fase 3)

Nodo	vCPU	RAM	Disco (dinámico)
bdd-nodo01 / 02 / 03 / 06	1	1536 MB	20 GB
bdd-nodo04 / 05	1	1024 MB	15 GB
bdd-cliente	1	1024 MB	10 GB

Total si todos los nodos corrieran simultáneamente: ~7 vCPU, ~9.25 GB RAM, ~120 GB disco (asignación dinámica, no se ocupa todo de inmediato).

D. Procedimiento paso a paso

Paso 1 — Verificar que el equipo host cumple los requisitos mínimos.

Antes de instalar nada, confirmamos que la PC física puede soportar VirtualBox y varias VMs Linux corriendo a la vez.

Requisitos mínimos recomendados:

- CPU de 4 núcleos físicos o más, con virtualización por hardware (Intel VT-x o AMD-V) habilitada en BIOS/UEFI.
- 16 GB de RAM como mínimo (32 GB ideal si se desea correr 4+ nodos simultáneos sin apagar otros).
- 120 GB de espacio libre en disco (SSD muy recomendado; HDD funcionará pero más lento).
- Windows 10/11 de 64 bits con permisos de administrador.

Paso 2 — Confirmar que la virtualización por hardware está habilitada.

Ver sección E para los comandos exactos.

Paso 3 — Definir el alcance temático del laboratorio.

Ya está definido por el plan de 19 fases (ver tabla de C.2). En esta fase solo lo confirmamos y lo documentamos como referencia.

Paso 4 — Diseñar el diagrama de arquitectura objetivo.

Usar el diagrama de la sección C como punto de partida; el estudiante debe poder explicarlo de memoria antes de avanzar.

Paso 5 — Definir convención de nombres.

`bdd-nodoNN` para nodos de datos, `bdd-cliente` para la estación de pruebas. Esta convención se usará en hostname, en el nombre de la VM dentro de VirtualBox, y en las carpetas de configuración.

Paso 6 — Definir el esquema de direccionamiento IP.

Ya documentado en C.3. Se reserva ahora aunque se configure hasta la Fase 6.

Paso 7 — Definir la estrategia de snapshots.

Cada fase terminada y verificada debe cerrar con un snapshot de VirtualBox nombrado `faseNN-completa` en cada VM involucrada. Esto permite retroceder si algo se rompe en una fase posterior, sin perder el trabajo previo.

Paso 8 — Crear la estructura de carpetas del proyecto en el host.

Ver comandos en la sección E.

Paso 9 — Validar el plan con el checklist de la sección H.

E. Comandos completos

E.1 Verificar virtualización por hardware habilitada (Windows, PowerShell)

Abrir PowerShell (no es necesario ser administrador para esta consulta):

```
PowerShell  
systeminfo
```

Buscar manualmente en la salida la sección **"Requisitos de Hyper-V"** (en inglés *Hyper-V Requirements*). Debe aparecer:

```
Text  
  
La virtualización habilitada en firmware: Sí
```

Si dice "No", la virtualización está deshabilitada en BIOS/UEFI y debe habilitarse manualmente reiniciando el equipo y entrando a la configuración de BIOS (la tecla varía por fabricante: Del, F2, F10, Esc).

E.2 Verificar RAM total y disponible

```
PowerShell  
systeminfo | findstr /C:"Memoria física total" /C:"Memoria físic"
```

(En equipos en inglés: `findstr /C:"Total Physical Memory"`)

E.3 Verificar espacio libre en disco

```
PowerShell  
wmic logicaldisk get size,freespace,caption
```

E.4 Verificar si Hyper-V está activo (posible conflicto con VirtualBox)

```
PowerShell  
bcdedit /enum | findstr hypervisorlaunchtype
```

Si el resultado es `hypervisorlaunchtype Auto`, Hyper-V está activo. VirtualBox 7.x moderno puede coexistir con Hyper-V, pero si en fases posteriores se presentan errores de arranque de VM (`VT-x is not available`), se puede desactivar así (requiere reinicio):

 PowerShell

```
bcdedit /set hypervisorlaunchtype off
```

*Nota: en este laboratorio normalmente **no será necesario** desactivar Hyper-V. Se documenta aquí solo como referencia para la sección de solución de problemas.*

E.5 Crear la estructura de carpetas del proyecto (en el host)

 PowerShell

```
mkdir C:\LabBDD
mkdir C:\LabBDD\VMs
mkdir C:\LabBDD\ISOs
mkdir C:\LabBDD\Scripts
mkdir C:\LabBDD\Snapshots-Notas
mkdir C:\LabBDD\Documentacion
```

Propósito de cada carpeta:

- `VMs`: aquí vivirán los discos virtuales (.vdi) de cada nodo.
- `ISOs`: imagen ISO de Ubuntu Server descargada (se usará en Fase 4).
- `Scripts`: scripts de configuración reutilizables que crearemos en fases futuras.
- `Snapshots-Notas`: bitácora en texto de qué representa cada snapshot.
- `Documentacion`: este archivo y los entregables de cada fase.

F. Verificación de funcionamiento

Esta fase no instala software, por lo que la “verificación” consiste en confirmar que la planeación está completa y es correcta:

1. `systeminfo` muestra virtualización habilitada en firmware → **Sí**.
2. El host tiene al menos 16 GB de RAM totales y al menos 120 GB libres en disco.
3. La carpeta `C:\LabBDD` existe con sus 5 subcarpetas.
4. El estudiante puede dibujar de memoria el diagrama de arquitectura objetivo (C.1 y C.2) y explicar el rol de cada nodo.

5. El estudiante puede explicar, sin ver el documento, la diferencia entre fragmentación horizontal y vertical.

G. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>systeminfo</code> muestra "Virtualización habilitada en firmware: No"	VT-x/AMD-V deshabilitado en BIOS/UEFI	Reiniciar, entrar a BIOS, habilitar "Intel Virtualization Technology" o "SVM Mode" (AMD), guardar y reiniciar
RAM total reportada es menor a la física instalada	RAM reservada por gráficos integrados o módulos defectuosos	Verificar en <code>Administrador de tareas</code> → <code>Rendimiento</code> → <code>Memoria</code> ; si la diferencia es muy grande, revisar configuración de memoria compartida con GPU integrada
Poco espacio libre en disco	Disco con muchos archivos/instalaciones previas	Liberar espacio antes de avanzar; los discos virtuales dinámicos crecerán con el uso, se necesita margen
Hyper-V activo y futura VM no arranca (error <code>VT-x is not available</code>)	Conflicto entre Hyper-V y VirtualBox en versiones antiguas	Actualizar VirtualBox a la versión 7.x más reciente (soporta modo Hyper-V) o desactivar Hyper-V con <code>bcdedit</code> (ver E.4)
Antivirus bloquea la creación de carpetas o adaptadores de red más adelante	Política de seguridad corporativa/educativa muy restrictiva	Solicitar excepción para <code>C:\LabBDD</code> y para el ejecutable de VirtualBox en el antivirus institucional

H. Checklist de validación

- ☐ Verifiqué que mi CPU soporta y tiene habilitada la virtualización por hardware.
- ☐ Confirmé que tengo al menos 16 GB de RAM (idealmente 32 GB).
- ☐ Confirmé que tengo al menos 120 GB libres en disco.
- ☐ Creé la estructura de carpetas `C:\LabBDD` con sus 5 subcarpetas.
- ☐ Puedo explicar la diferencia entre replicación física y lógica.
- ☐ Puedo explicar la diferencia entre fragmentación horizontal, vertical e híbrida.
- ☐ Entiendo y puedo reproducir el diagrama de arquitectura objetivo (7 nodos, roles, IPs).
- ☐ Entiendo por qué se eligió una red Host-Only y no NAT o Bridged.

- ☐ Tengo claro el plan de crecimiento gradual (de 2 nodos en Fase 3 a 6–7 nodos en Fase 16).
 - ☐ Entiendo la estrategia de snapshots por fase.
-

I. Preparación para la siguiente fase

La **Fase 1: Instalación de VirtualBox** requerirá:

- Permisos de administrador en Windows.
- Conexión a internet para descargar el instalador desde el sitio oficial de Oracle VirtualBox.
- La estructura de carpetas `C:\LabBDD` ya creada (Paso 8 de esta fase).
- Haber resuelto cualquier conflicto de virtualización detectado en esta fase (BIOS/UEFI o Hyper-V).

No se necesita descargar todavía la ISO de Ubuntu Server; eso corresponde a la Fase 4.

Preguntas teóricas para estudiantes

1. ¿Qué diferencia existe entre la replicación física y la replicación lógica en un sistema de bases de datos distribuidas, y por qué este laboratorio planea nodos distintos para cada una?
2. ¿Qué significa “transparencia de distribución” y qué tipos de transparencia existen (fragmentación, ubicación, replicación, concurrencia)?
3. ¿Por qué es conveniente separar, desde el diseño, los nodos dedicados a fragmentación horizontal de los nodos dedicados a replicación?
4. Según el teorema CAP, ¿qué compromisos debe enfrentar quien diseña una arquitectura distribuida como la planeada en este laboratorio?
5. ¿Qué ventajas y qué limitaciones tiene usar una sola computadora física para simular un sistema distribuido de varios nodos, comparado con usar varias máquinas físicas reales?

Ejercicios prácticos

1. Verificar en su propia computadora si la virtualización por hardware (VT-x/AMD-V) está habilitada, documentando el procedimiento con los comandos de la sección E y el resultado obtenido (puede incluir capturas de pantalla).
2. Elaborar un diagrama propio de la arquitectura objetivo (puede usar papel, draw.io o cualquier herramienta) representando los 7 nodos propuestos, sus roles, IPs planeadas y la fase en la que se activa cada uno.

3. Calcular cuántos nodos podría correr su computadora simultáneamente sin saturar la RAM, usando la fórmula: $(\text{RAM total} - 4 \text{ GB reservados para el sistema operativo host}) / \text{RAM asignada por VM}$. Justificar el resultado con sus propios datos de hardware.

Reto adicional para alumnos avanzados

Proponer y justificar técnicamente una arquitectura alternativa que incluya un **octavo nodo de tipo proxy/balanceador** (por ejemplo, conceptualmente similar a ProxySQL o MariaDB MaxScale), indicando en qué fase del plan de 19 fases se debería introducir y qué problema resolvería que los 7 nodos actuales no resuelven por sí solos. No es necesario implementarlo, solo justificarlo por escrito.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Verificación de requisitos de hardware	20%	El estudiante presenta evidencia (capturas/comandos) de que su equipo cumple los mínimos, o documenta correctamente las limitaciones encontradas
Diagrama de arquitectura	25%	El diagrama incluye todos los nodos, roles, IPs y es consistente con el plan de 19 fases
Comprensión conceptual	25%	Responde correctamente las 5 preguntas teóricas con vocabulario técnico adecuado
Justificación de decisiones de diseño	15%	Explica con claridad por qué se eligió Host-Only Adapter, por qué se separan roles por nodo, y por qué se usa una estrategia de snapshots
Reto avanzado (si aplica)	15%	La propuesta del nodo proxy está técnicamente justificada y ubicada coherentemente en el plan de fases